

Name Muneeb Afridi

Id 5279/1169 -2024 – C

Assignment 1

Basic Structure of a Computer System

A computer system is made up of different components that work together to process data and produce meaningful information. The main parts are:

- CPU (Central Processing Unit)
- Memory (Primary Storage)
- Input/Output (I/O) Devices

1. Central Processing Unit (CPU)

Definition

The CPU is the brain of the computer. It controls all operations and processes data.

Components of CPU

- Control Unit (CU)
 - Directs all operations in the computer
 - Controls data flow between CPU, memory, and I/O devices
- Arithmetic Logic Unit (ALU)
 - Performs calculations (addition, subtraction, etc.)
 - Handles logical operations (AND, OR, NOT)
- Registers

- Small storage areas inside CPU for fast processing

Role of CPU

- Executes instructions
- Processes data
- Controls all system activities

2. Memory (Primary Memory)

Definition

Memory is used to store data and instructions temporarily or permanently.

Types of Memory

a) RAM (Random Access Memory)

- Temporary (volatile)
- Data is lost when power is off

b) ROM (Read Only Memory)

- Permanent (non-volatile)
- Stores system instructions (boot program)

Role of Memory

- Stores data and instructions for CPU
- Holds intermediate and final results
- Provides quick access to data

3. Input Devices

Definition

Input devices are used to enter data into the computer.

Examples

- Keyboard
- Mouse
- Scanner

Role

- Convert user input into machine-readable form
- Send data to CPU for processing

4. Output Devices

Definition

Output devices display or produce the processed information.

Examples

- Monitor
- Printer
- Speakers

Role

- Convert processed data into human-readable form
- Present results to the user

Conclusion

A computer system works by taking input, processing it using the CPU, storing it in memory, and producing output. All components are interconnected and depend on each other for proper functioning.

Q2

No.	Computer Architecture	Computer Organization
1	Focuses on what the system does	Focuses on how the system works
2	Visible to programmer	Not visible to programmer
3	Deals with instruction set, data types	Deals with hardware, circuits, control signals
4	Example: Instruction set like ADD, SUB	Example: How ADD is implemented in ALU
5	Changes affect program execution	Changes do not affect programs directly
6	Conceptual design	Physical implementation
7	Concerned with system functionality	Concerned with system efficiency

Q3

Von Neumann Architecture

Definition

The John von Neumann architecture is a computer design in which data and instructions are stored in the same memory and processed by the CPU step by step.

Main Components

1. Memory Unit

- Stores both data and instructions
- Each location has a unique address
- CPU reads and writes data from here

2. Central Processing Unit (CPU)

a) Control Unit (CU)

- Controls all operations
- Fetches and decodes instructions

b) Arithmetic Logic Unit (ALU)

- Performs calculations (add, subtract)
- Performs logical operations (AND, OR)

c) Registers

- Small, fast memory inside CPU
- Example: Program Counter (PC), Instruction Register (IR)

3. Input Unit

- Takes data from user (keyboard, mouse)
- Sends it to memory/CPU

4. Output Unit

- Shows results (monitor, printer)

Working (Fetch–Decode–Execute Cycle)

The CPU works in a cycle:

1. Fetch

- Instruction is taken from memory
- Address comes from Program Counter (PC)
- Stored in Instruction Register (IR)

2. Decode

- Control Unit understands the instruction
- Decides what action is needed

3. Execute

- ALU or CPU performs the operation
- Result is stored in memory or shown as output

Cycle Representation

Fetch → Decode → Execute

Q4

Instruction Cycle (Step-by-Step)

The Instruction Cycle is the process by which the CPU executes a program instruction. It consists of three main stages:

1. Fetch

- The CPU fetches the instruction from memory
- The address of the instruction is stored in the Program Counter (PC)
- The instruction is loaded into the Instruction Register (IR)

After fetching, the PC is updated to point to the next instruction

2. Decode

- The Control Unit decodes the instruction in the IR
- It determines:
 - What operation to perform
 - Which data is needed

3. Execute

- The CPU performs the operation
- The ALU carries out calculations if needed
- The result is stored in the Accumulator (AC) or memory

Cycle Representation

Fetch → Decode → Execute

Role of Important Registers

1. Program Counter (PC)

- Holds the address of the next instruction
- Automatically increases after each fetch
- Ensures instructions are executed in sequence

Example: If PC = 100, the CPU fetches instruction at address 100

2. Instruction Register (IR)

- Stores the current instruction being executed
- Sends the instruction to the Control Unit for decoding

Acts like a “temporary holder” of the instruction

3. Accumulator (AC)

- Stores intermediate and final results of operations
- Works closely with the ALU

Example:

If you add $5 + 3 \rightarrow$ result (8) is stored in AC

Simple Flow Summary

1. PC $\rightarrow$ gives address
2. Instruction fetched $\rightarrow$ stored in IR
3. Instruction decoded
4. Execution performed
5. Result stored in AC

Q5

Assembly Language

Definition

Assembly Language is a low-level programming language that uses mnemonic codes (symbols) instead of binary to write instructions for a computer.

It is closer to machine language but easier for humans to understand.

Features of Assembly Language

- Uses mnemonics (e.g., ADD, SUB, MOV) instead of 0s and 1s
- Machine dependent (different for each processor)
- Requires an assembler to convert into machine code
- Provides direct control over hardware

- Faster execution compared to high-level languages
- Difficult to learn compared to high-level languages

Structure of an Instruction

An assembly instruction usually has the following format:

[Label] Opcode Operand(s) ; Comment

Explanation

- **Label (optional)**
 - Used to identify a memory location
 - Example: LOOP:
- **Opcode (Operation Code)**
 - Specifies the operation to perform
 - Example: MOV, ADD, SUB
- **Operands**
 - Data or memory locations used in the instruction
 - Example: registers like AX, BX
- **Comment (optional)**
 - Used to explain the instruction
 - Starts with ;

Example

MOV AX, 5 ; Move value 5 into register AX
ADD AX, BX ; Add BX to AX

